

Using Negative Emotional Intensity Scores to Improve Emotional Awareness in Large Language Models

Tino Saarenpää

Abstract—Large language models (LLMs) are increasingly used as conversation systems, where their responses should not only be factual, but also emotionally suitable for the given context. This project studies whether having access to an explicit score of negative emotional intensity would increase the model’s ability to self-reflect in terms of the emotional suitability of the output. Scoring of such text can trivially be done using a model fine-tuned on a supervised dataset to classify input into a scalar value. The goal of more emotionally aware output is motivated by the lackluster performance of state-of-the-art LLMs with regards to handling of cases with high emotional intensity. Moreover, by building this functionality separate from the model, we can utilize it with any arbitrarily chosen model given that it supports tool calling.

I. INTRODUCTION

THE goal of this project is to answer whether LLMs would be able to better take the user’s negative emotional intensity into account during inference if they were given an explicit score. Pretrained LLMs are generally trained into chatbots by fine-tuning them on a supervised instruction dataset and by aligning them via reinforcement learning from human feedback (RLHF) [1]. RLHF by definition makes the model less accurate with regards to the datasets it has been trained on, since after all, during RLHF the model is rewarded on behavior that matches the preferences of its trainers, not the ground truth of the datasets. Inference providers may additionally use prompt engineering by defining a custom system prompt to further alter the models they are offering. Prompt engineering is convenient, because it allows tailoring general-purpose LLMs to all kinds of applications and workflows without necessarily having to fine-tune them [2].

II. BACKGROUND

By utilizing RLHF and prompt engineering we obtain models that are more often than not overly helpful and verbose [3], both of which can be inferred as inappropriate depending on the context. Despite the astonishing reasoning abilities of the most powerful models, even they struggle with formulating their responses in a way that would take user frustration effectively into account. Models should be able to better adapt to the current situation by, for example, answering in a clear and concise way without constantly apologizing if the user is clearly irritated and wants to get on with their task.

As stated before, chatbots are trained to be helpful, which, on top of other things, includes taking care of user frustration. As a matter of fact, Claude Code, the agent built on top

of Anthropic’s flagship Claude chatbot, has a sophisticated frustration detection system built-in right into the agent [4]. If nothing else, this should serve as clear evidence of how hard it is to fix the problem by purely using fine-tuning and reinforcement learning if even the top competitors are unable to do so without the help of external software.

III. PROJECT

The novelty of this project lies in the fact that the proposed system is independent of the underlying model. I propose giving models access to a tool that returns the negative emotional intensity of a string using a separately trained classifier. The hypothesis is that the LLM can make use of the tool and its score to better understand the user’s emotional state during reasoning.

More specifically, a base model was fine-tuned into a linear classifier classifying input text into a scalar value indicating negative emotional intensity ranging from 0.0 to 1.0. The model was fine-tuned using a public dataset which had been downloaded and then relabeled separately with the help of an LLM. After training, the classifier was provided to two different LLMs as a tool with which they could retrieve the negative emotional intensity for the latest message. The tool could be invoked by having the LLM return an appropriate tool call instead of a final answer, followed by the client executing the tool and providing the resulting score back to the model. The model could then use the score to finish its reasoning and generate the final response. The experiment was conducted by providing the models with a predefined list of queries and comparing their responses with and without access to the tool.

I elected to use Massive Text Embedding Benchmark’s Toxic conversations dataset consisting of 50,000 entries for training the classifier. Each entry has a textual input and a boolean label indicating whether the input is considered toxic or not. The dataset fits well into our use case due to it having a disproportionately large amount of text containing high negative emotional intensity as compared to most other datasets where such samples would have been pruned away. A random sample of 10,000 entries was taken from the dataset, each of which was then relabeled using DeepSeek V4.1 Flash (low). The model was called in batches of 10 entries with a system prompt containing the criteria of how the scores should be calculated. The original labels were discarded in favor of the scores given by the model.

The classifier used in the project, BERT, is widely regarded as one of the key breakthrough models that significantly

advanced research in artificial intelligence following its publication in 2018 [5]. What makes BERT particularly useful for our purpose is its relatively small size in terms of parameters. A small model makes training faster and more accessible to people with less powerful hardware. It also helps with avoiding overfitting in cases where there is not as much of a need for a more sophisticated model [6].

DeepSeek V4.1 Flash was chosen for both annotating the dataset and running the experiment, since it is the only paid model I had API access for. With 552 billion parameters [7], the model should be more than capable of calling and utilizing the tool correctly. Additionally, frontier models such as the DeepSeek V4.1 Flash are frequently used in the real world, meaning that the results are rather applicable to some concrete tasks and workflows.

Cohere’s North Mini Code model was later added to the list of models used in the experiment. With a size of 30 billion parameters [8], it is able to represent models magnitudes of times smaller than the first model. The hypothesis is that the result of the experiment strongly depends on the size of the model due to larger models’ better reasoning skills.

The model was trained for three epochs using a learning rate of 5×10^{-5} with a batch size of 16. I chose the fused AdamW optimizer and enabled bfloat16 floating-point precision. The training parameters were chosen using one of the three following ways: sticking with the default value, picking a widely recommended value or using the maximum value limited by the GPU, a measly GeForce GTX 1060 with 6GB of VRAM. The dataset size was chosen as a round number that was sufficiently large for training an effective classifier while remaining small enough to keep the cost of annotation manageable. It is assumed that the quality of the model is not necessarily limited by the size of the dataset, but rather the quality of the dataset. The model’s performance is more likely to suffer due to the labels being dependent on subjective interpretation rather than a ground truth which the model could realistically learn.

IV. RESULTS

The results are conflicting to say the least. The larger model, DeepSeek V4.1 Flash (max), called the tool on each message. Based on the output alone, it seems like having a concrete score presented to the model does indeed make it self-reflect more by having the model explicitly mention the user’s frustration more often. Yet, it does not seem that the model used this information to adapt the emotional tone of its output to that of the input, quite the contrary: the messages were on average about twice the size. Moreover, the model was already quite capable of recognizing user emotion as is as evidenced by the responses without the tool, so the benefit of having access to one could be considered minimal at best.

One noteworthy exception occurred when the model responded to a query that was otherwise benign with the exception of it being written in all capital letters as shown in Table I. The classifier correctly rated the message with a fairly high score of 0.5166, yet the model with no access to the tool did not seem to catch this emotion encoded in the message.

TABLE I
AN EXAMPLE OF THE OUTPUT GENERATED BY DEEPSEEK V4.1 FLASH WITH AND WITHOUT HAVING ACCESS TO THE TOOL

Prompt	DO THE ZERO:RED IEMS COME WITH A 10 OHM ADAPTER?
Score	0.5166
w/o tool	Yes — the Truthear x Crinacle Zero:RED retail package includes a 10Ω impedance adapter . It’s meant to be used inline with the cable to add a bass boost (and it can also lower overall volume slightly). If you’re buying used/open-box, verify it’s included, since those adapters often go missing.
w/ tool	Short answer: yes. The Truthear × Crinacle Zero:RED ships with a 10Ω impedance adapter included in the box. A few details: - It’s built into one of the two included cables — you get a standard cable and a second cable with the 10Ω adapter, so you don’t need to buy one separately. - Using the adapter raises the bass by roughly +2.5–3 dB below 300 Hz, which is the warmer, bass-boosted tuning you’ll see in graphs labeled “with 10Ω adapter.” - Without it, the set is noticeably more neutral/bass-light. So if you were expecting it and couldn’t find one, check the second cable — the adapter section is easy to miss since it just looks like a slightly thicker connector segment on the cable. If you’re buying used, that second cable is the piece to make sure is still included.

Now, the other model could be considered to be at odds with the information it was given: on one hand the model does not give much weight to the capitalization, but on the other hand it has acquired an explicit score indicating high levels of negative emotional intensity. In the end, however, the output of this model does not seem to be more empathetic towards the user as compared to the other model, indicating that access to the tool did not significantly alter the output.

The tool performed even worse on Cohere’s North Mini Code model. The model is unable to correctly answer the base query to begin with, since it has no internet access nor has it been trained on any dataset containing the relevant information. This leads to the model either refusing to respond or hallucinating. Additionally, the model decided not to call the tool on multiple occasions, and only called it when the query contained clear signs of frustration. When the tool was called, the model seems to have used it for explicitly mentioning being able to sense the frustration, sometimes going as far as using the exact score in the response.

V. CONCLUSION

In this project, we investigated the possibility of using a negative emotional intensity classifier to help LLMs better adapt the tone of their responses during reasoning. Despite the rather small sample size and subjective evaluation of the experiment, it is quite clear that the results are not satisfactory to say the least. Regardless of the size, the models were already well capable of recognizing human emotion. Providing the score to the models did not substantially improve or even change the output of the models when compared to those instances where the model did not have access to the tool. Even then, the models kept being overly verbose, discarding user frustration as a sign to keep the response short and relevant. The score of the tool mainly confused the models, especially those of smaller size.

VI. AI USAGE

LLMs were not used for writing or editing this document or for writing the code used in this project. AI was used for annotating the dataset, training the classifier, and for generating the responses for the predefined queries.

VII. SOURCE CODE

The source code for the project is available at <https://github.com/ymcx/affective-computing-project>

REFERENCES

- [1] DataStudios. (2025, May) How ai chatbots work: Complete guide to models, training, deployment, and system integration. [Online]. Available: <https://www.datastudios.org/post/how-ai-chatbots-work-complete-guide-to-models-training-deployment-and-system-integration>
- [2] PromptLayer. (2026) What is a system prompt? [Online]. Available: <https://www.promptlayer.com/glossary/system-prompt>
- [3] H. Wang, W. Xiong, T. Xie, H. Zhao, and T. Zhang. (2024, May) Interpretable preferences via multi-objective reward modeling and mixture-of-experts. [Online]. Available: <https://rlhflow.github.io/posts/2024-05-29-multi-objective-reward-modeling>
- [4] Agnost AI. (2025) We dug into claude code’s source code. anthropic built a full frustration detection system. [Online]. Available: <https://agnost.ai/blog/claude-code-frustration-detection-source-code>
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [6] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. (2020) Scaling laws for neural language models. [Online]. Available: <https://openai.com/index/scaling-laws-for-neural-language-models/>
- [7] Artificial Analysis. (2026) Deepseek v4.1 flash (max) intelligence, performance & price analysis. [Online]. Available: <https://artificialanalysis.ai/models/deepseek-v4-1-flash>
- [8] Cohere. (2026, Jun.) Introducing north mini code: Cohere’s first model for developers. [Online]. Available: <https://cohere.com/blog/north-mini-code>